Overclock e largura de memória no RK3229

Teste antes/depois, overclock de CPU, DDR a 786 MHz e as defesas que impedem brick

Este guia mede o sistema **antes** de mexer em qualquer coisa, aplica o overclock em duas etapas (CPU e depois DRAM) e repete **exatamente** os mesmos testes no fim. Só assim o ganho é atribuível a cada mudança. Vale para box3, box5 e box6. **O box2 fica de fora**: kernel 5.15 sem `rk3228_dmc.ko` e é o DNS/DHCP da casa.

Resposta curta sobre brick: **sim, existem seis mecanismos independentes** que impedem transformar o box em peso de papel — e nenhum deles depende de você acertar de primeira. A seção 1 explica cada um; a seção 9 é o passo a passo de recuperação.

> **NOTA** Todo número de referência deste documento vem de medições já feitas na frota (box3/box5/box6). O box3 não estava alcançável quando o guia foi gerado, então nada aqui foi remedido agora.

Glossário — o vocabulário que o resto do guia usa

Termo	O que é, na prática
SoC	O chip único (RK3229): 4x Cortex-A7, VPU, controlador de memória, tudo dentro.
DTB	*Device Tree Blob*. Arquivo binário que descreve o hardware para o kernel: quais periféricos existem, em que endereço, com que clock e que tensão. O kernel não descobre nada sozinho; se não está no DTB, não existe.
Overlay (dtbo)	Remendo aplicado **em cima** do DTB no boot. Não substitui o DTB: acrescenta ou altera nós. É assim que se adiciona um OPP sem recompilar o kernel.
OPP	*Operating Performance Point*: um par frequência + tensão. O kernel só usa frequências que estejam na tabela de OPPs.
DVFS	*Dynamic Voltage and Frequency Scaling*: subir a frequência e, junto, a tensão (mais clock precisa de mais voltagem).
Governor	A política que escolhe o OPP: `ondemand` sobe/desce conforme a carga; `performance` fica no máximo.
PLL	Circuito que gera o clock. `sclk_ddrc` é a saída que alimenta a DRAM.
DMC	*Dynamic Memory Controller*: o bloco que agenda os acessos à DRAM. No Linux aparece como um dispositivo *devfreq* com governador próprio.
VPU / rkvdec	O decodificador de vídeo por hardware. É o trabalho real do box3 (1080p60) e a razão de a banda de memória importar.
Regulador vdd_arm	Fonte que alimenta o cluster de CPU. É PWM e tem teto de **1.4 V** gravado no device tree.

1. As seis defesas anti-brick

"Brickar" é deixar a placa sem conseguir bootar nada. No RK322x isso é difícil por construção: a ROM de boot do SoC é **somente leitura** e sempre procura um cartão SD antes da eMMC. As camadas abaixo são independentes — se uma falhar, a seguinte segura.

#	Defesa	Onde vive	O que ela salva
1	Overlay aplicado em RAM	script de boot	O DTB em disco nunca é reescrito. Basta tirar a linha do `armbianEnv.txt`.
2	Fallback do u-boot	u-boot	Se o overlay não aplica, ele boota com o DT original em vez de travar.
3	Boot pelo SD primeiro	BootROM do SoC	Um cartão Multitool sempre entra, mesmo com a eMMC morta.
4	Console serial 115200	pads TX/RX/GND	Você vê onde parou e interrompe o boot para agir.
5	Maskrom + rkdeveloptool	BootROM + USB OTG	Último recurso: grava direto na eMMC sem sistema nenhum.
6	Watchdog + limite de tensão	kernel / regulador	Travou, reinicia; e o regulador recusa qualquer coisa acima de 1.4 V.

1.1 O overlay nunca toca no disco

O arquivo `/boot/armbianEnv.txt` é só texto. No boot, o u-boot lê a linha `user_overlays=`, carrega o `.dtbo`, aplica sobre uma **cópia em memória** do DTB e entrega essa cópia ao kernel. O DTB gravado na eMMC permanece intacto. Consequência prática: **um overlay errado nunca corrompe nada permanente** — no pior caso o box boota sem ele, ou boota com um clock que não existe e você remove a linha.

1.2 O que acontece se o overlay for inválido

O carregador tenta aplicar o fragmento; se o alvo não existir (por exemplo `&cpu0_opp_table` ausente) ou o blob estiver corrompido, a aplicação falha e o u-boot segue com o DT original. Você perde o overclock, não o boot. É o mesmo princípio de um "modo de segurança" automático.

1.3 A ROM de boot sempre tenta o SD antes

O RK322x tem ordem de boot fixa: SD → eMMC → USB (maskrom). Por isso o cartão do **Multitool** é a chave-mestra: com ele no slot, o box boota um Linux completo mesmo que a eMMC esteja com o bootloader quebrado. É a diferença entre "não boota" e "não tem sistema".

1.4 O console serial é o seu olho

Sem serial você adivinha. Com serial você lê o log do u-boot e vê exatamente onde parou: overlay rejeitado, OPP inválido, kernel panic. O box3 tem os três pads (TX, RX, GND) em nível **3.3 V**, **115200 8N1**. Ligação: RK TX → adaptador RX, RK RX → adaptador TX, GND ↔ GND.

1.5 O que estas defesas NÃO cobrem

Corte de energia durante `apt upgrade` do kernel ou durante um `dd` na eMMC pode corromper a partição de boot — aí você cai no nível 3 da recuperação (Multitool). E nenhuma defesa de software salva hardware: curto, fonte errada ou calor extremo. O overclock por OPP não entra nessa lista porque a tensão é limitada pelo regulador.

2. Preparação — 15 minutos que valem um box

2.1 Backup do único arquivo que quebra o boot

```
sudo cp /boot/armbianEnv.txt /boot/armbianEnv.txt.bak-$(date +%F)
cat /boot/armbianEnv.txt        # anote o que já estava lá
```

O `armbianEnv.txt` guarda **tudo** que decide o boot: `overlays=`, `user_overlays=`, `fdtfile=`, `extraargs=`. É o primeiro lugar a olhar quando algo para de bootar.

2.2 Testar o dtbo ANTES de rebootar

Esta é a defesa mais subestimada: dá para validar o overlay na máquina ligada, sem reboot. Primeiro confira que o alvo existe no DTB base; depois aplique o overlay num arquivo temporário.

```
fdtget /boot/dtb/rockchip/rk322x-box.dtb /__symbols__ cpu0_opp_table
fdtoverlay -i /boot/dtb/rockchip/rk322x-box.dtb \
           -o /tmp/teste.dtb /boot/overlay-user/cpuoc.dtbo
dtc -I dtb -O dts /tmp/teste.dtb 2>/dev/null | grep -A2 opp-1488000000
```

Se o `fdtoverlay` sair sem erro e o último comando mostrar o nó de 1488 MHz com `opp-microvolt`, o overlay é aplicável. Se der erro, **não reboote**: o problema está no DTS, não no box.

```
sudo apt install device-tree-compiler stress-ng mbw -y
```

2.3 Backup da região de boot da eMMC

O que pode ser corrompido por energia é a região do bootloader (os primeiros ~32 MB) e as partições de boot da eMMC. Backup disso é pequeno e cabe em qualquer cartão; o backup da eMMC inteira (~7 GB) só se houver espaço. Faça pelo Multitool, que enxerga a eMMC como `/dev/mmcblk2`:

```
dd if=/dev/mmcblk2      of=/mnt/backups/boot-region.img bs=1M count=32
dd if=/dev/mmcblk2boot0  of=/mnt/backups/emmc-boot0.img  bs=1M
# eMMC inteira, se o SD tiver espaço:
dd if=/dev/mmcblk2 of=/mnt/backups/emmc-completa.img bs=4M status=progress
```

2.4 Cartão Multitool pronto

A imagem está em `~/Arquivo/rk322x-multitool-backup/multitool.img.xz` (63 MB). Os links oficiais do Multitool saíram do ar; esse backup local é a fonte. Grave no cartão antes de mexer, não depois:

```
xz -dc ~/Arquivo/rk322x-multitool-backup/multitool.img.xz \
  | sudo dd of=/dev/sdX bs=4M conv=fsync status=progress
```

Troque `/dev/sdX` pelo cartão — **confira com** `lsblk` **antes**. Escrever no disco errado é o único brick real que existe neste guia.

2.5 Console serial

```
sudo picocom -b 115200 /dev/ttyUSB0      # 3.3 V, 8N1, sem flow control
```

Diagnóstico rápido: **zero bytes** = fio errado ou sem sinal; **lixo** = baud errado; ASCII legível = achou. No Armbian o Linux loga em `ttyS2` com auto-login root; o u-boot também usa 115200. Android/vendor usa 1500000 — não é o caso aqui.

2.6 Checklist antes do primeiro reboot

	Item	Comando de conferência
☐	Backup do armbianEnv	`ls /boot/armbianEnv.txt.bak-*`
☐	dtbo validado com fdtoverlay	`fdtoverlay -i ... -o /tmp/t.dtb ...`
☐	Overlay aparece no dtc do teste	`dtc -I dtb -O dts /tmp/t.dtb | grep 1488`
☐	Multitool gravado e testado	bootar dele uma vez
☐	Serial conectado e lendo	aparece log ao ligar
☐	Fonte estável, sem nobreak ruim	—

3. Rodada A — o sistema base, medido antes de qualquer overlay

Regras que valem para as duas rodadas: **boot frio** (depois de um clamp em runtime o clock reportado mente), **5 min de idle**, governor `performance` nos dois lados, **mesmos comandos e duração**, 3 repetições e mediana. O A7 tem ruído grande.

```
mkdir -p ~/bench && cd ~/bench
echo 'GOVERNOR="performance"' | sudo tee /etc/default/cpufrequtils
sudo systemctl restart cpufrequtils
```

3.1 Identidade e clocks

```
uname -r | tee A-base.txt
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq | tee -a A-base.txt
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies | tee -a A-base.txt
sudo cat /sys/kernel/debug/clk/clk_summary | grep -E "sclk_ddrc|ddr" | tee -a A-base.txt
```

`scaling_cur_freq` vem em kHz. O `clk_summary` é o **único** lugar que diz a verdade sobre a DRAM — no stock o esperado é `330000000` (330 MHz), não 786.

3.2 CPU — 4 threads e 1 thread, 60 s cada

```
stress-ng --cpu 4 --cpu-method matrixprod --timeout 60s --metrics --quiet \
  | tee -a A-base.txt
stress-ng --cpu 1 --cpu-method matrixprod --timeout 60s --metrics --quiet \
  | tee -a A-base.txt
```

O número que importa é **bogo ops/s** (maior = melhor). 1 thread isola o ganho de clock; 4 threads mostra se aparece throttle (se o ganho de 4 threads for menor que o de 1, a térmica está cortando).

3.3 Inteiro/hash e banda de memória

```
/usr/bin/time -f "%e s" sh -c 'dd if=/dev/zero bs=1M count=512 2>/dev/null \
  | sha256sum' 2>&1 | tee -a A-base.txt
mbw -n 50 256 2>&1 | tee -a A-base.txt
```

O `mbw` imprime três testes (MEMCPY, DUMB, MCBLOCK). Anote **MEMCPY**: é o que mais se aproxima do padrão de acesso do VPU. Stock ≈ 472 MiB/s com DDR a 330 MHz.

3.4 Temperatura e sanidade

```
cat /sys/class/thermal/thermal_zone0/temp    # milésimos de °C: 58000 = 58 °C
# durante o stress, em outro terminal:
while :; do cat /sys/class/thermal/thermal_zone0/temp; sleep 1; done | sort -n | tail -1
sudo dmesg | grep -iE "invalid rate|opp|ebusy|-16" | tee -a A-base.txt
```

O último comando **tem que sair vazio**. Qualquer linha ali é clock rejeitado ou OPP inválida — e esses erros aparecem tanto no boot quanto em runtime.

3.5 Decode real — só no box3

```
journalctl -u tv-player -f | grep -iE "drop|fps|warn"
```

É o trabalho de verdade do box3: 1080p60 por 10 min. Esperado no stock: 0 drops, ~60 fps, ~75 °C. Qualquer medida de CPU deste pipeline feita com `fakesink` é inválida: ele força a cópia do quadro e infla o custo em ~5×.

4. Overclock de CPU

4.1 Por que 1488 MHz é o teto

Não é térmico, é de projeto. O driver `rockchip_cpuclk` tem uma tabela **fixa** de frequências aceitas: **1296 / 1392 / 1488 MHz**. Pedir 1440 devolve `Invalid rate : 1440000000 for cpuclk` (EBUSY -16) — não é falha de tensão, é o PLL recusando. E o regulador `vdd_arm` é PWM com **máximo de 1.4 V** definido no device tree; qualquer OPP que peça mais que isso é rejeitado. Resultado: 1488 MHz roda **no teto de tensão** e 1.5 GHz é fisicamente inviável nesta plataforma.

> **ATENÇÃO** É por isso que overclock aqui não "queima" o SoC: não existe caminho de software para subir a tensão acima de 1.4 V. O risco real é térmico e de estabilidade, não de fritar o chip.

4.2 Atalho — box5/box6 (dtbo stock já presente)

```
ls /boot/dtb/overlay/ | grep -E "cpu|ddr"     # existe rk322x-cpu-hs.dtbo?
sudo sed -i 's/^overlays=.*/overlays=cpu-hs/' /boot/armbianEnv.txt
```

Entrega 1296/1392 MHz (+16%). O `cpu-hs` é um overlay oficial do Armbian rk322x — o "hs" é *high speed*. Para chegar a 1488, use o caminho custom.

4.3 Custom até 1488 — método do box3

```
cat > cpuoc.dts <<'EOF'
/dts-v1/;
/plugin/;
/ {
  compatible = "rockchip,rk3228", "rockchip,rk3229";
  fragment@0 {
    target = <&cpu0_opp_table>;
    __overlay__ {
      opp-1296000000 { opp-hz = /bits/ 64 <1296000000>;
                       opp-microvolt = <1325000 1325000 1400000>; };
      opp-1392000000 { opp-hz = /bits/ 64 <1392000000>;
                       opp-microvolt = <1350000 1350000 1400000>; };
      opp-1488000000 { opp-hz = /bits/ 64 <1488000000>;
                       opp-microvolt = <1400000 1400000 1400000>; };
    };
  };
};
EOF
dtc -I dts -O dtb -o cpuoc.dtbo cpuoc.dts
sudo mkdir -p /boot/overlay-user
sudo cp cpuoc.dtbo /boot/overlay-user/
sudo sed -i 's/^user_overlays=.*/user_overlays=cpuoc/' /boot/armbianEnv.txt
```

Leitura do DTS: `target = <&cpu0_opp_table>` diz em qual nó o remendo entra; cada `opp-hz` é a frequência em Hz; `opp-microvolt` é o trio (mínimo, típico, máximo) em microvolts. O terceiro valor nunca pode passar de 1400000.

> **ATENÇÃO** Nunca inclua OPP de 1440 no overlay. O `ondemand` tenta esse passo e o kernel passa a spammar `-16` no `dmesg`. Se o overlay falhar ao aplicar, o u-boot restaura o DT original — o fallback da seção 1.2.

4.4 Governor

```
echo 'GOVERNOR="performance"' | sudo tee /etc/default/cpufrequtils
sudo systemctl restart cpufrequtils
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

Sem governor fixo o clock passeia e a comparação vira ruído. `/etc/default/cpufrequtils` é lido no boot, então persiste.

5. Largura de memória (DDR)

5.1 Por que a DRAM fica em 330 MHz (e por que o teto é 786)

O controlador de memória tem seus próprios OPPs, e o overlay `ddr3-*` só marca esses pontos como `okay`. Sem nenhum overlay `ddr3-*` no `armbianEnv.txt`, o único ponto habilitado é o mais baixo: **330 MHz**. Além disso, o driver `rk3228_dmc.ko` existe no DTB mas **não autoloada** — sem ele não há governador de devfreq para subir a frequência. O nome "800" é comercial; a OPP real mais alta da tabela é **786 MHz**.

Os overlays oficiais: `rk322x-ddr3-534`, `-660` e `-786`. O que você escreve no env é o sufixo (`ddr3-800`), e o script prefixa `rk322x-`.

```
sudo sed -i 's/^overlays=.*/overlays=cpu-hs ddr3-800/' /boot/armbianEnv.txt
echo rk3228_dmc | sudo tee /etc/modules-load.d/rk3228_dmc.conf
ls /sys/class/devfreq/          # confirme o nó (deve ser "dmc")
```

```
sudo systemctl edit --force --full dmc-perf.service <<'EOF'
[Unit]
Description=DMC performance governor
After=multi-user.target

[Service]
Type=oneshot
ExecStart=/bin/sh -c 'echo performance > /sys/class/devfreq/dmc/governor'
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl enable --now dmc-perf.service
```

`/sys/class/devfreq/dmc/governor` é o botão do devfreq; `performance` trava na frequência máxima. Se o nó tiver outro nome, ajuste o caminho — o `ls /sys/class/devfreq/` da primeira caixa mostra.

6. Rodada B — depois do overclock (mesmos comandos)

```
cd ~/bench
sudo reboot
# ... esperar 5 min de idle ...
cp A-base.txt B-oc.txt
```

Repita **exatamente** os passos 3.1 a 3.5 gravando em `B-oc.txt`. Conferências que só fazem sentido aqui:

```
grep "^cpu " /proc/cpuinfo                          # 4 núcleos no clock novo?
sudo dmesg | grep -iE "invalid rate|opp|ebusy|-16"    # VAZIO
sudo cat /sys/kernel/debug/clk/clk_summary | grep sclk_ddrc   # 786000000
systemctl is-active tv-player tv-receiver             # box3: serviços de pé
```

> **NOTA** Quer atribuir cada ganho a uma mudança? Rode uma rodada intermediária **só com CPU** (antes do passo 5) e outra **depois da DDR**. É o que separa "+16% de CPU" de "+97% de RAM" em vez de um bolo só.

7. Critérios de aprovação

Item	Aprova	Reprova
`stress-ng` 180 s, 4 threads	passa, sem reboot	crash, hang, watchdog
Temperatura de pico	**< 85 °C**	throttle (cai 1488 → 1392)
`dmesg`	limpo	`Invalid rate`, -16, OPP
Decode 1080p60, 10 min (box3)	**0 drops**, ~60 fps	drops, 403, travamento
SHA-256	+16% (1392) / +24% (1488)	ganho zero = clock não aplicou
`mbw` MEMCPY	**~932 MiB/s**	continua 472 = DDR não subiu

8. Números de referência e estado da frota

Configuração	CPU	SHA-256	DDR	MEMCPY
stock	1200 MHz	153 MB/s	330 MHz	472 MiB/s
cpu-hs	1392 MHz (+16%)	178 MB/s	330 MHz	472 MiB/s
cpuoc	1488 MHz (+24%)	~190 MB/s	786 MHz	**932 MiB/s (+97%)**

Térmica: o box3 fez **75 °C** a 1488 no stress (10 °C de folga até o throttle). Um dissipador de R$ 10–20 tira 10–15 °C e elimina o pior caso: decode 40 Mbit + CPU 100% juntos = 89 °C, throttle e drops no `kmssink`.

8.1 Frota

box	CPU	DDR	Observação
box1	1200 (não mexer)	786	control-plane k3s
box2	—	—	**NÃO TOCAR** (DNS/DHCP, kernel 5.15)
box3	**1488**	**786**	já no teto; é a bancada
box5	1392	786	pode ir a 1488 com o `cpuoc.dtbo`
box6	1488	786	feito 24/08

9. Runbook de recuperação

Três níveis, do mais provável ao mais extremo. Antes de tudo: **ligue a serial e leia o log**. A mensagem de erro quase sempre diz qual nível é.

Nível 1 — boota, mas o overclock não aplicou

Sintoma: o sistema sobe, mas `scaling_available_frequencies` não tem 1488, ou `sclk_ddrc` continua 330.

```
grep -E "overlays|user_overlays" /boot/armbianEnv.txt
ls -l /boot/overlay-user/cpuoc.dtbo
sudo dmesg | grep -iE "overlay|fdt|invalid rate"
```

Causas comuns: nome do `.dtbo` diferente do escrito no env, arquivo em lugar errado, ou o kernel não ter o símbolo `cpu0_opp_table`. Corrija o arquivo/texto e reboote — nada foi danificado.

Nível 2 — não boota (loop, tela preta, sem rede)

O caminho mais seguro é bootar o **Multitool** pelo SD e editar o `armbianEnv.txt` da eMMC de fora:

```
# no Multitool (boota do SD; eMMC = mmcblk2)
mkdir -p /mnt/emmc
mount /dev/mmcblk2p1 /mnt/emmc
nano /mnt/emmc/boot/armbianEnv.txt     # remova user_overlays= / overlays=cpu-hs ddr3-800
sync && umount /mnt/emmc && reboot
```

Alternativa sem Multitool: interromper o u-boot pela serial (spam de Ctrl-C ou tecla na janela de "Hit any key") e bootar manualmente. Se o u-boot responder, o box **não está brickado** — é só configuração.

> **ATENÇÃO** Boot loop com o watchdog ativo não é brick: o box reinicia sozinho a cada falha. O watchdog existe justamente para não deixar a placa pendurada; o preço é o ciclo.

Nível 3 — eMMC corrompida (nem o Multitool monta a raiz)

```
# no Multitool
xz -t /mnt/images/IMAGEM.img.xz                # integridade (lento no A7, ~3 min)
xz -dc /mnt/images/IMAGEM.img.xz | dd of=/dev/mmcblk2 bs=512K iflag=fullblock oflag=direct
blkid /dev/mmcblk2p1                           # deve dar LABEL=armbi_root TYPE=ext4
```

Depois, restaure o backup da região de boot se você tiver um. O primeiro boot refaz o resize do rootfs e pede a criação de usuário pela serial.

Nível 4 — último recurso: maskrom

Só quando nem o SD boota. A BootROM entra em maskrom se a eMMC não responder; force fechando o clock da eMMC (ou pelo pad de maskrom, conforme a placa), conecte o USB OTG ao PC e use o `rkdeveloptool`:

```
sudo rkdeveloptool ld                # lista o dispositivo em maskrom
sudo rkdeveloptool db loader.bin     # carrega o loader na RAM
sudo rkdeveloptool wl 0 imagem.img   # grava na eMMC
sudo rkdeveloptool rd                # reinicia
```

> **ATENÇÃO** O maskrom é o caminho **menos** confiável no RK322x: já houve caso de "flashou mas nunca bootou" por loader incompatível. Prefira sempre o Multitool. O maskrom serve para ressuscitar, não para o dia a dia.

9.1 Sintoma → causa → ação

Sintoma	Causa provável	Ação
`Invalid rate 1440000000`	OPP de 1440 no overlay	remover esse nó e recompilar
Boot loop após reboot	overlay inválido ou typo no env	Nível 2 (Multitool/serial)
Uptime cai a cada poucos minutos	watchdog reiniciando por hang de OPP	remover o OPP mais alto
`sclk_ddrc` continua 330	overlay ddr3 ausente ou módulo não carregou	ver seção 5
Clock sobe e volta	governor ondemand	fixar performance
Throttle e drops	térmica	dissipador; conferir pasta
Sem vídeo após mexer na DDR	DDR instável (raro em 786)	reverter só a linha ddr3-800

10. Reverter tudo

```
sudo cp /boot/armbianEnv.txt.bak-AAAA-MM-DD /boot/armbianEnv.txt
sudo rm -f /boot/overlay-user/cpuoc.dtbo \
           /etc/modules-load.d/rk3228_dmc.conf \
           /etc/systemd/system/dmc-perf.service
sudo reboot
```

Antes de rebootar, confira o resultado com a validação de dtbo da seção 2.2: o DTB base sem o overlay aplica limpo. Se você removeu o `ddr3-800`, a DRAM volta a 330 MHz — o que **não** quebra o 1080p60, só reduz a banda.

> **ATENÇÃO** Trocar o governor da DDR em runtime **não re-trava o PLL para cima** depois de um clamp userspace. A/B ao vivo dá número falso. Sempre: boot frio → mede.